sagemath

mahmood

<2022-06-18 Sat 14:15:00>

note
video demonstrating sagemath+org-mode workflow:
`https://www.youtube.com/watch?v=cd4_O3TqArs`

sagemath is an open source alternative to Mathematica written in python, it preparses python code to allow for extra syntactic sugars
its basically a python interface to a collection of mathematical software like Maxima/sympy

1 plotting

1.1 area below functions

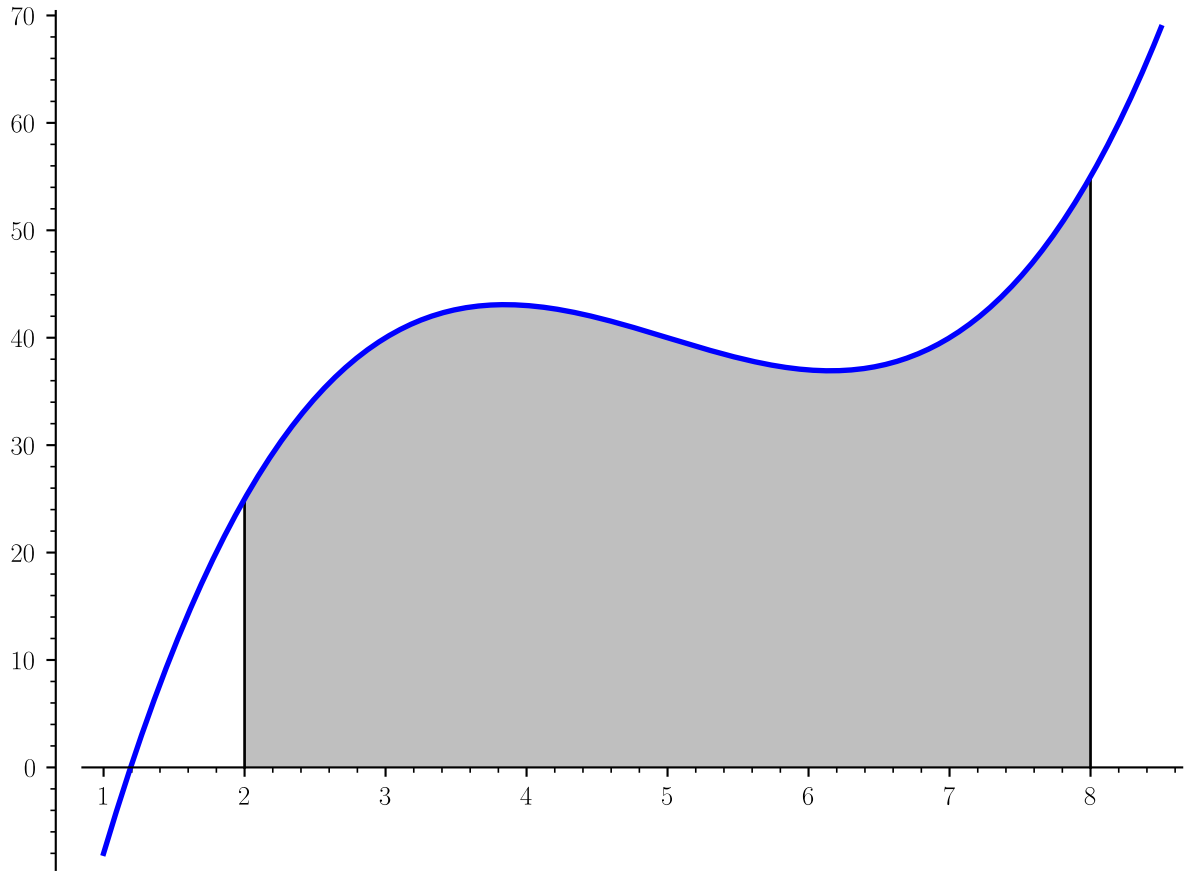
```
interval = [1, 8.5]
interval_fill = [2, 8]
f(x) = (x-3)*(x-5)*(x-7)+40
import os.path

try: _ = open_left
except: open_left = False
try: _ = open_right
except: open_right = False

if not open_left:
  my_plot = line([(interval_fill[0],0),(interval_fill[0],f(
      interval_fill[0]))], color='black')
```

```
if not open_right:
  my_plot += line([(interval_fill[1], 0),(interval_fill[1], f(
     interval_fill[1]))], color='black')
my_plot += plot(f, (interval_fill[0], interval_fill[1]), fill=
   True) + plot(f, (interval[0], interval[1]), thickness=2)
my_plot.save(filename=os.path.expanduser(filename), transparent
   =True)
print(os.path.expanduser(filename))

open_left = False
open_right = False
```

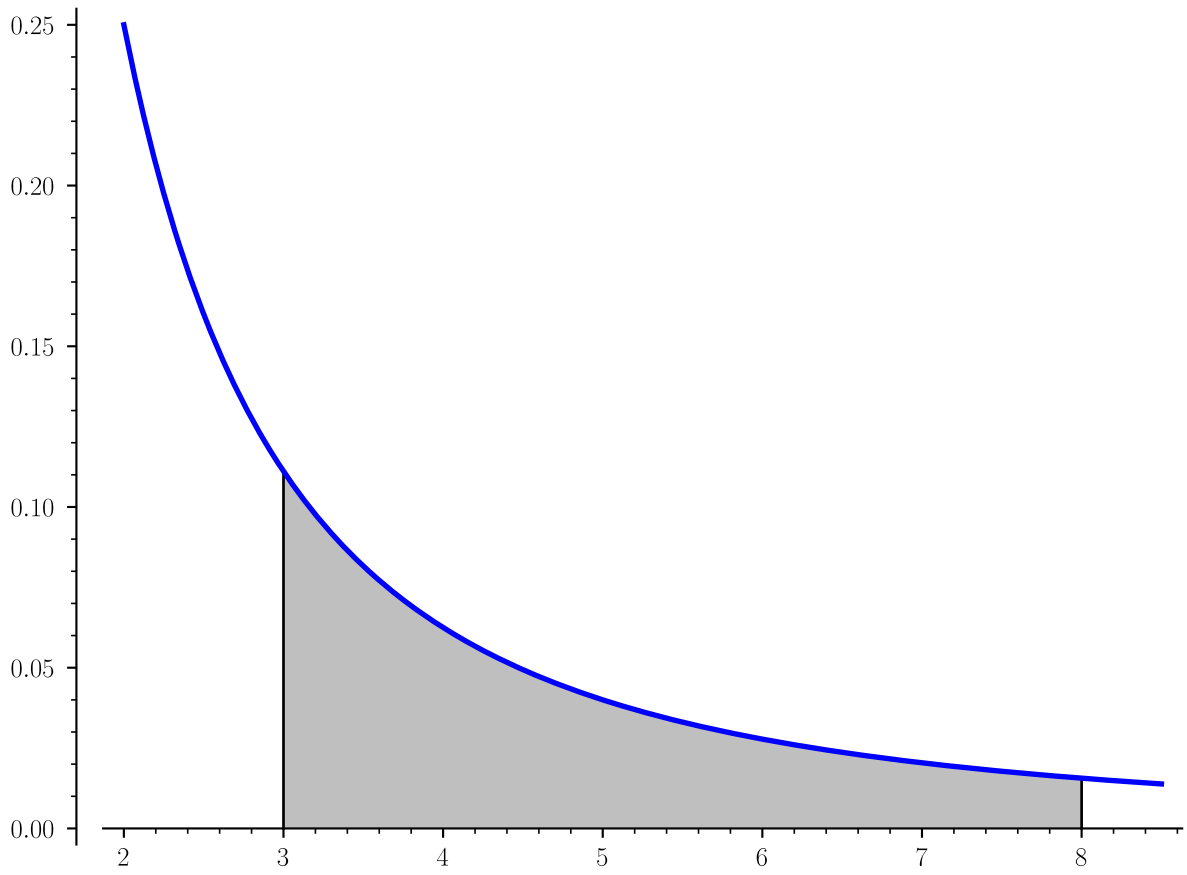



```
interval = [2, 8.5]
interval_fill = [3, 8]
f(x) = 1/x^2
import os.path
```

```
try: _ = open_left
except: open_left = False
try: _ = open_right
except: open_right = False

if not open_left:
  my_plot = line([(interval_fill[0],0),(interval_fill[0],f(
      interval_fill[0]))], color='black')
if not open_right:
  my_plot += line([(interval_fill[1], 0),(interval_fill[1], f(
      interval_fill[1]))], color='black')
my_plot += plot(f, (interval_fill[0], interval_fill[1]), fill=
    True) + plot(f, (interval[0], interval[1]), thickness=2)
my_plot.save(filename=os.path.expanduser(filename), transparent
    =True)
print(os.path.expanduser(filename))

open_left = False
open_right = False
```



1.2 solid of revolution

consider the solid of revolution of the function e^x rotated around the y axis

```
my_plot = revolution_plot3d(e^x, (x,0,1), show_curve=True,
   parallel_axis='z', figsize=5)
```

1.3 parametric 3d plot

```
my_plot = parametric_plot3d([cos(x),sin(x),x/10], (x,0,4*pi),
   color='red', viewer='threejs')
```

1.4 more

```
x,y,z = var('x,y,z')
f(x,y,z)=x^2+y^2+z^2/4
my_plot = Graphics()
for k in [1,2,3,4]:
    my_plot += implicit_plot3d(f(x,y,z)==k,(x,-2,2),(y,-2,2),(z
        ,-5,5),opacity=0.4)
```

1.5 animation

```
def build_frame(t):
    e = parametric_plot3d([sin(x-t), 0, x], (x, 0, 2*pi), color
        ='red')
    m = parametric_plot3d([0, -sin(x-t), x], (x, 0, 2*pi),
        color='green')
    return e + m
frames = [build_frame(t) for t in (0, pi/32, pi/16, .., 2*pi)]
my_plot = animate(frames).interactive()
```

```
sines = [plot(c*sin(x), (-2*pi,2*pi), color=Color(c,0,0), ymin
    =-1, ymax=1,transparent=False) for c in sxrange(0,1,.1)]
a = animate(sines)
a.save(os.path.expanduser(f))
print(f)
```

file:///Users/mahmooz/brain/out/IRCODD.gif

```
sin_frames = []
circ_frames = []
circ_x = -1.5 # offset center of the circle
circ_y = 0
for i in srange(0,2*pi,0.2):
    singraph = point((i,sin(i)), color="green", size=50)
    singraph += plot(sin(x),(0,2*pi), xmin=0, xmax=7, ymin=-1,
        ymax=1, figsize=[2,2], axes=False)
    unitcircle = point((cos(i)+circ_x,sin(i)+circ_y), color="
        green", size=50)
    unitcircle += circle((circ_x,circ_y),1, color="blue",
        figsize=[2,2], axes=False)
```

```
    sin_frames.append(singraph)
    circ_frames.append(unitcircle)
A1 = animate(sin_frames)
A2 = animate(circ_frames)
(A1 + A2).save(os.path.expanduser(f))
print(f)
```

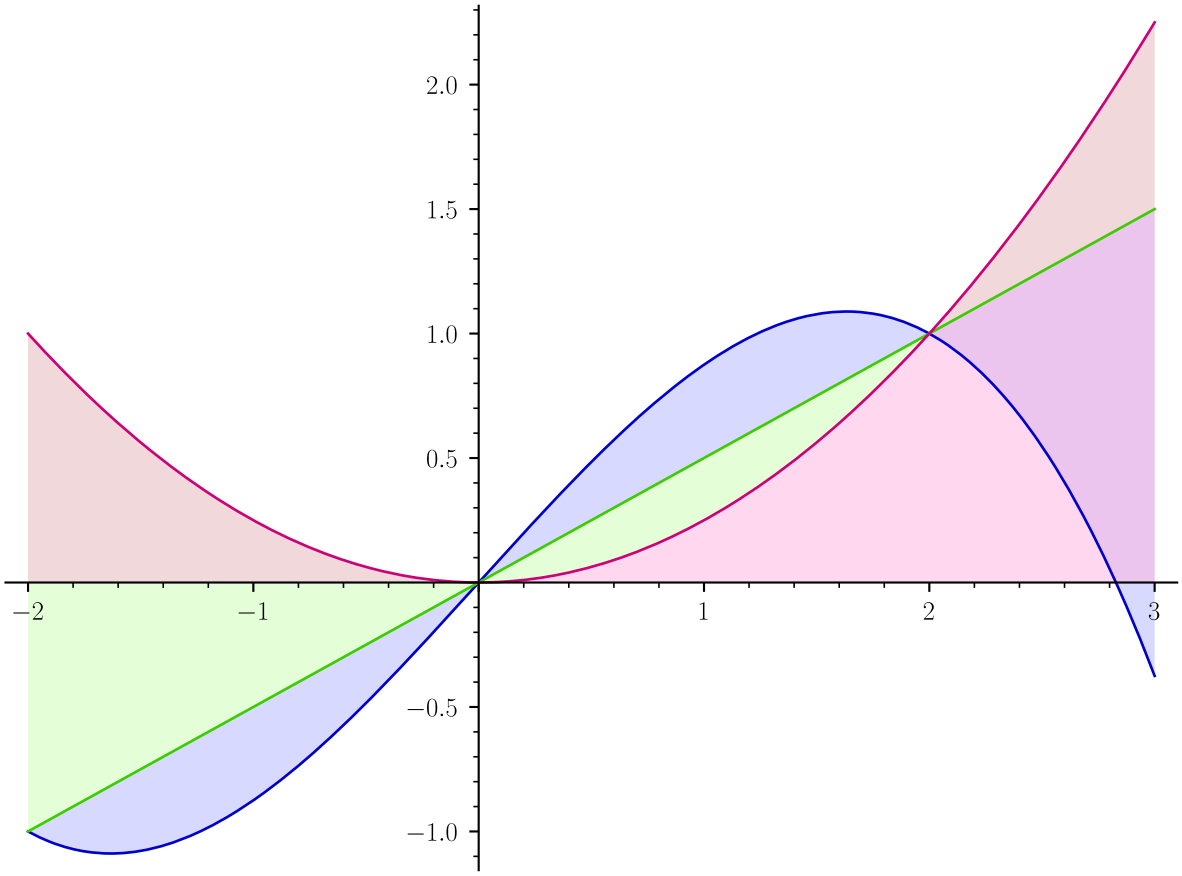
file:///Users/mahmooz/brain/out/Sg63BY.gif

```
f(x) = 5*x - x^2/8 + x^3/1200
df(x) = diff(f(x),x)
max = 100
p = plot(f(x),x,0,max)
lblp = text("$y = " + latex(f(x)) + "$",[40,100],fontsize=14)
lbldp = text("$y' = " + latex(df(x)) + "$",[40,80],fontsize=14,
   rgbcolor='#006000')
ga = []
for argx in srange(0,max,5):
  dp = plot(f(argx) + (df(argx)*(x-argx)),x,0,max,color
     ="#006000")
  xp = point((argx,f(argx)),rgbcolor='#800000')
  ga.append(p+lblp+lbldp+dp+xp)
a = animate(ga,ymin=0,ymax=max,axes_labels=['x','y'],fontsize
   =12,figsize=(4,3))
a.save(os.path.expanduser(out_file))
print(out_file)
```

file:///Users/mahmooz/brain/out/yAyHE1.gif

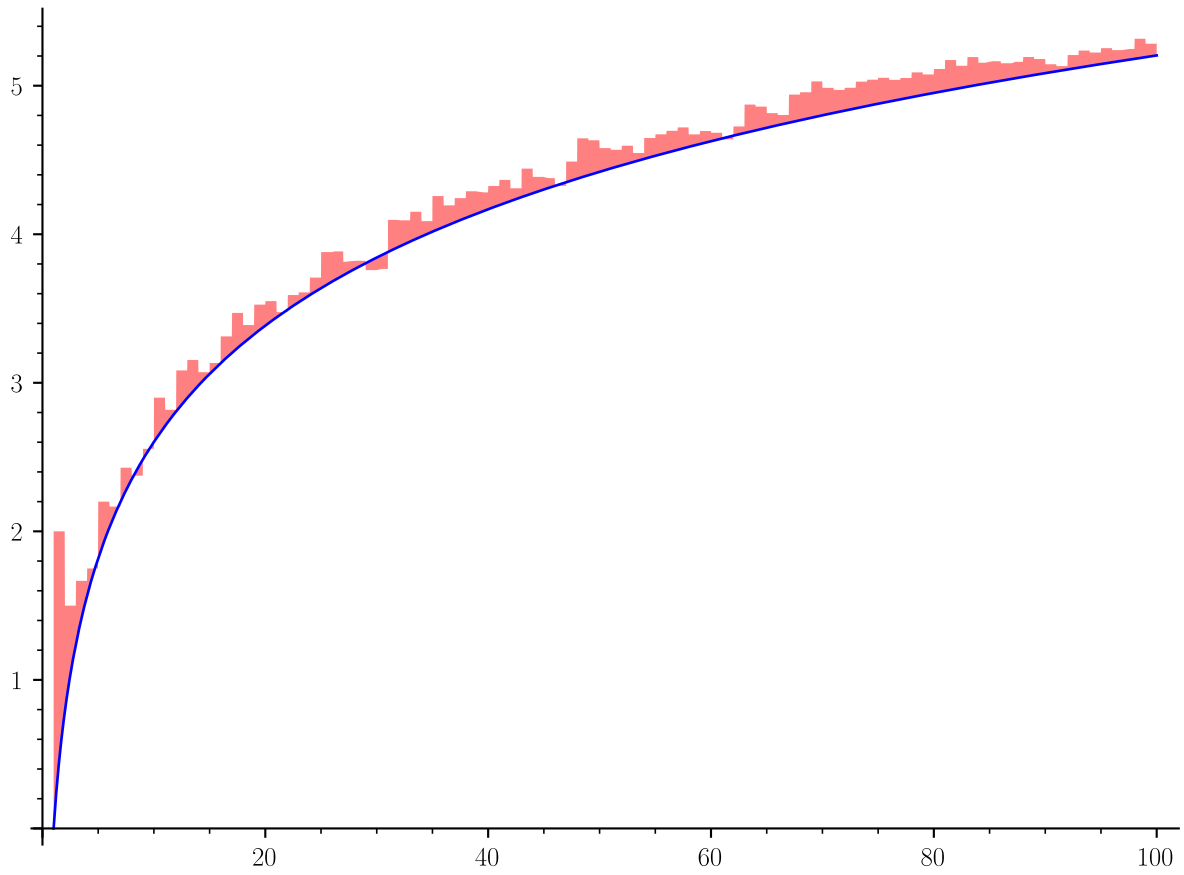
1.6 area between functions

```
f(x) = -1/8*x^3+x
g(x) = 1/2*x
h(x) = 1/4*x^2
my_plot = plot([f,g,h],-2,3,fill={0:g,1:h,2:0})
```

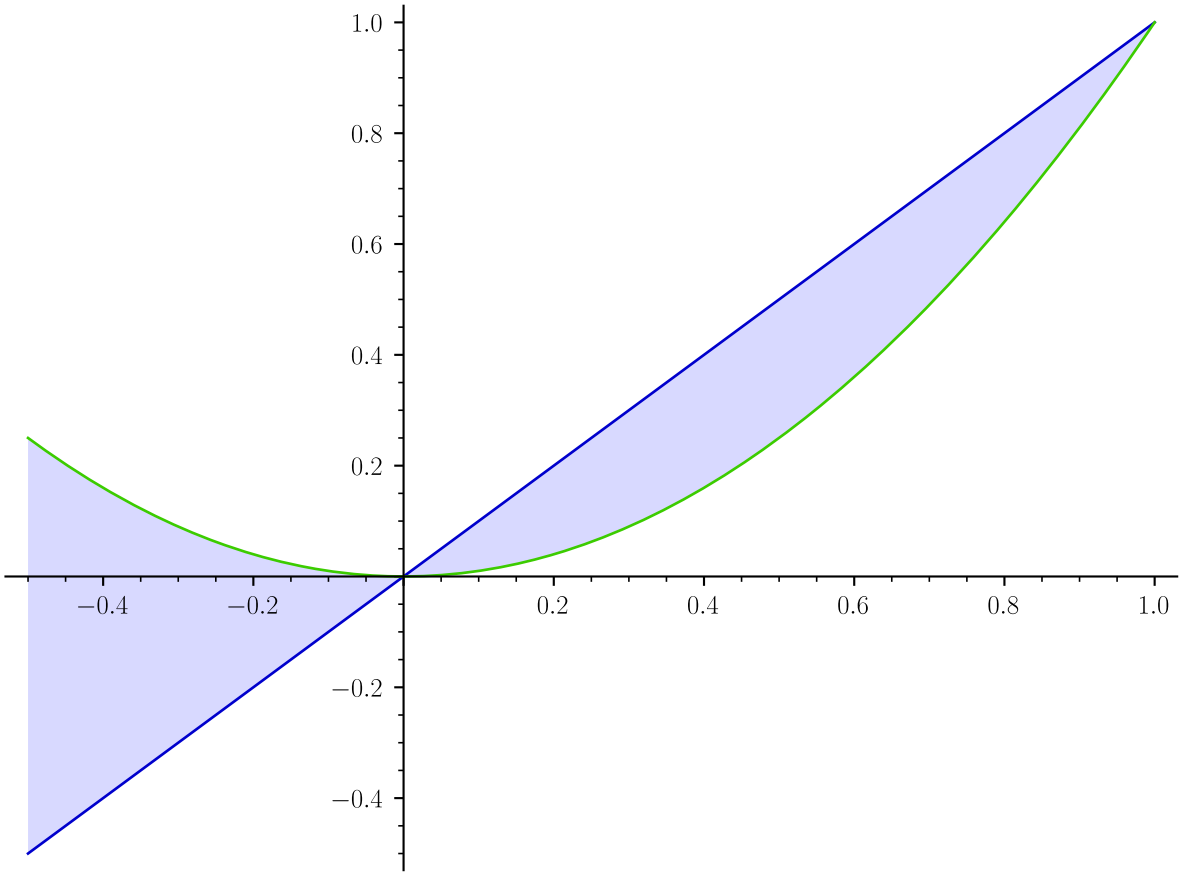


1.7 more filling

```
my_plot = plot(1.13*log(x), 1, 100, fill = lambda a: nth_prime(
    a)/floor(a), fillcolor = 'red')
```




```
#plot(x,(x,0,1),fill=x^2)
my_plot = plot([x,x^2],(x,-0.5,1),fill={0:[1]})
```



2 examples

2.1 recursive sum with sage

this also shows a limitation of sagemath's, i thought something as simple as a quick 1-liner
using the sum function would do the job

```
def ev(self, x):
  if x == 0:
    return 1
  else:
    return 1 + sum(self(j) for j in range(x))
f = function("f", nargs=1, eval_func=ev)
```

```
[f(i) for i in range(12)]
```

[1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048]

2.2 recursive sum but pure python

```
def f(n):
  if n == 0:
    return 1
  else:
    total_sum = 0
    for i in range(n):
      total_sum += f(i)
    return total_sum + 1
print([f(i) for i in range(12)])
```

```
[1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048]
```

2.3 vectors of variables

```
b = list(var("b", n=8, latex_name='\\overline{x}'))
b[0]=123
show(b)
```

[123, b1, b2, b3, b4, b5, b6, b7]

2.4 indexed variables

consider the following summation:

$$\sum_{i=1}^{k} a_i \cdot d = \frac{k}{2}[2a_1 + (k-1)d]$$

here a_i is always dependant on the index i and we cant define a variable number of variables
so we define a as a function of i

```
a = function("a")
d = var("d")
j, i, k = var("j, i, k", domain="integer")
S = sum(a(j)*d, j, 1, k+1)
S(j=10)
```

d*sum(a(10), 10, 1, k + 1)

2.5 symbolic functions

just like you can define a symbolic variable, you can also define yet-unknown functions!

```
T = var('T')
a = function('a')
R = var('R')
v = var('v')
b = var('b')
p = R*T/v - a(T)/v/(v+b)
diff(p, T)
```

R/v - diff(a(T), T)/((b + v)*v)

2.6 boolean functions

```
from sage.crypto.boolean_function import BooleanFunction
f = BooleanFunction([0,0,0,1,0,1,1,1])
f.truth_table()
f.linear_structures().list()
```

```
from sage.crypto.boolean_function import BooleanFunction
B.<a, y> = BooleanPolynomialRing()
f = BooleanFunction(-a*-y)
f2 = BooleanFunction([1,0,0,0])
f.algebraic_normal_form()
f2.truth_table()
f.truth_table()
[[f(i), f2(i)] for i in range(4)]
```

3 limitations

consider the following equation:

$$t = x^2 + 5$$

we use this kind of equation when integrating by substituion

unfortunately sagemath cant derivate this with respect to t as the following snippet gives an error

```
t = x^2 + 5
t.derivative(t)
```

but weirdly enough it can handle the inverse integral...

```
t = x^2 + 5
t.integrate(t)
```

1/2*(x^2 + 5)^2

3.1 wrong limit

consider the limit:

$$\lim_{x\to\infty} \frac{1}{\tan^2 x + 1}$$

which is indeterminate, but sage apparently doesnt know that cuz it gives a result:

```
f = 1/(1+tan(x)^2)
f.limit(x=oo)
```

which is obviously incorrect..

3.2 i found this problem on a discord server dedicated to maths

$$\sum_{i=0}^{10}\left[\prod_{j=i}^{1}(10-j)\right]=108$$

i tried replicating it in sagemath but unfortunately sagemath cant handle this either

```
j,i = var('j,i')
from sage.calculus.calculus import symbolic_product
sum(symbolic_product(10-j, j, i, 1), i, 0, 10)
```

this block of code makes perfect sense to me but idk why it doesnt work

3.3 limit of sequence

consider the following limit of sequence:

$$\lim_{n\to\infty}\frac{3n}{\sqrt{2n^4-n^2}}+\frac{3n}{\sqrt{2n^4-4n^2}}+\frac{3n}{\sqrt{2n^4-9n^2}}+\cdots+\frac{3n}{\sqrt{2n^4-n^4}}$$

i tried implementing this in sagemath but it cant solve it properly:

```
k,n = var('k,n')
s = sum((3*n)/sqrt(2*n^4-k^2*n^2), k, 1, n)
s.limit(n=Infinity)
```

3*limit(n*sum(1/sqrt(-k^2*n^2 + 2*n^4), k, 1, n), n, +Infinity)

note

i asked how to deal with this on sagemath's forums

however, it was able to handle a simpler sequence:

$$\lim_{n\to\infty} \frac{n}{2}\left(\frac{3}{(n+1)^2} + \frac{3}{(n+2)^2} + \cdots + \frac{3}{(n+n)^2}\right)$$

```
%display plain
k,n = var('k,n')
s = n/2 * sum(3/(n+k)^2, k, 1, n)
s.limit(n=Infinity)
%display unicode_art
k,n = var('k,n')
s = n/2 * sum(3/(n+k)^2, k, 1, n)
s.limit(n=Infinity)
```

3/2*limit(n*(harmonic$_{\text{number}}$(2*n, 2) - harmonic$_{\text{number}}$(n, 2)), n, +Infinity)
3/4

weirdly tho we only get the correct answer when using `unicode_art`

we try sympy:

```
from sympy import limit_seq, Sum
from sympy.abc import n, k
limit_seq(n/2 * Sum(3/(n+k)**2, (k, 1, n)), n)
```

this takes forever to run, you can interrupt and rerun the last line and it would output some weird result

im really thinking about switching to mathematica.

4 discrete math

4.1 defining sets

consider the following set:

$$A = \{x \in \mathbb{R} \mid -10 \leq x \leq 10\}$$

this set is infinite as there is an infinite amount of real numbers in the interval $[-10, 10]$, to define such a set in sagemath we use `RealSet`:

```
A = RealSet.closed(-10, 10)
A.contains(5)
A.contains(-5)
A.contains(10)
A.contains(20)
```

im not sure how to do the same with other number fields like integers or rationals

5 linear algebra

for reference: refcard

5.1 matrices

5.1.1 guassian elimination

this method reduces matrices with guassian elimination method step by step

```
def gauss_method(M,rescale_leading_entry=False):
    """Describe the reduction to echelon form of the given
       matrix of rationals.

    M  matrix of rationals e.g., M = matrix(QQ, [[..], [..],
       ..])
    rescale_leading_entry=False boolean make the leading
       entries to 1's

    Returns: None. Side effect: M is reduced. Note: this is
       echelon form,
```

```
    not reduced echelon form; this routine does not end the
       same way as does
    M.echelon_form().

    """
    num_rows=M.nrows()
    num_cols=M.ncols()
    ld(M)

    col = 0  # all cols before this are already done
    for row in range(0,num_rows):
        # ?Need to swap in a nonzero entry from below
        while (col < num_cols
               and M[row][col] == 0):
            for i in M.nonzero_positions_in_column(col):
                if i > row:
                    print(" swap row",row+1,"with row",i+1)
                    M.swap_rows(row,i)
                    ld(M)
                    break
            else:
                col += 1

        if col >= num_cols:
            break

        change_flag = False
        # Now guaranteed M[row][col] != 0
        if (rescale_leading_entry
            and M[row][col] != 1):
            fm(r"take {1/M[row][col]} times row {row+1}")
            # print(" take",1/M[row][col],"times row",row+1)
            M.rescale_row(row,1/M[row][col])
            ld(M)
            change_flag=False
        for changed_row in range(row+1,num_rows):
            if M[changed_row][col] != 0:
                change_flag=True
                factor=-1*M[changed_row][col]/M[row][col]
                print(fr'take ${latex(factor)}$ times row {row
                   +1} plus row {changed_row+1}')
```

```
                M.add_multiple_of_row(changed_row,row,factor)
        if change_flag:
            ld(M)
            col +=1
```

example usage:

```
print("first matrix:")
R.<a> = PolynomialRing(QQ)
MS = MatrixSpace(SR, 3, 4)
m = MS.matrix([[1,a,-1,2],
              [2,-1,a,5],
              [1,10,-6,1]])
gauss_method(m)
print("second matrix:")
m = matrix(QQ, 4, [
    3, 0, 3, -2,
    -3, 3, 0, 2,
    0, -2, 2, 0,
    0, 3, -3, 0
])
gauss_method(m)
```

first matrix:

$$\begin{bmatrix} 1 & a & -1 & 2 \\ 2 & -1 & a & 5 \\ 1 & 10 & -6 & 1 \end{bmatrix}$$

take -2 times row 1 plus row 2
take -1 times row 1 plus row 3

$$\begin{bmatrix} 1 & a & -1 & 2 \\ 0 & -2\,a-1 & a+2 & 1 \\ 0 & -a+10 & -5 & -1 \end{bmatrix}$$

take $-\frac{a-10}{2\,a+1}$ times row 2 plus row 3

$$\begin{bmatrix} 1 & a & -1 & 2 \\ 0 & -2\,a-1 & a+2 & 1 \\ 0 & 0 & -\frac{(a+2)(a-10)}{2\,a+1}-5 & -\frac{a-10}{2\,a+1}-1 \end{bmatrix}$$

second matrix:

$$\begin{bmatrix} 3 & 0 & 3 & -2 \\ -3 & 3 & 0 & 2 \\ 0 & -2 & 2 & 0 \\ 0 & 3 & -3 & 0 \end{bmatrix}$$

take 1 times row 1 plus row 2

$$\begin{bmatrix} 3 & 0 & 3 & -2 \\ 0 & 3 & 3 & 0 \\ 0 & -2 & 2 & 0 \\ 0 & 3 & -3 & 0 \end{bmatrix}$$

take $\frac{2}{3}$ times row 2 plus row 3
take -1 times row 2 plus row 4

$$\begin{bmatrix} 3 & 0 & 3 & -2 \\ 0 & 3 & 3 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & -6 & 0 \end{bmatrix}$$

take $\frac{3}{2}$ times row 3 plus row 4

$$\begin{bmatrix} 3 & 0 & 3 & -2 \\ 0 & 3 & 3 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

5.1.2 matrices with variables

to be able to use symbolic variables in matrices we use the symbolic ring `SR`
cosnider the following:

```
v1 = matrix(SR, 4, 1, [1,2,0,0])
v2 = matrix(SR, 4, 1, [0,1,0,0])
v3 = matrix(SR, 4, 1, [var(f'x{i}') for i in range(4)])
ld(v1.augment(v2).augment(v3), v1.augment(v2).augment(v3).
   echelon_form())
```

$$\begin{bmatrix} 1 & 0 & x_0 \\ 2 & 1 & x_1 \\ 0 & 0 & x_2 \\ 0 & 0 & x_3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

this isnt really the result we're looking for, because the correct result will most certainly contain the symbolic variables $x_{0\to 3}$
to get proper results we use `PolynomialRing`

```
R = PolynomialRing(QQ, 4, 'x')
v1 = matrix(R, 4, 1, [1,2,0,0])
v2 = matrix(R, 4, 1, [0,1,0,0])
v3 = matrix(R, 4, 1, R.gens())
ld(v1.augment(v2).augment(v3), v1.augment(v2).augment(v3).
   echelon_form())
```

$$\begin{bmatrix} 1 & 0 & x_0 \\ 2 & 1 & x_1 \\ 0 & 0 & x_2 \\ 0 & 0 & x_3 \end{bmatrix} \begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & -2x_0 + x_1 \\ 0 & 0 & x_2 \\ 0 & 0 & x_3 \end{bmatrix}$$

now this, this is a proper result, cheers!
to get the same functionality but with other Rings/Fields we can use `MatrixSpace`
here im using the `SR` field (short for Symbolic Ring)

```
R.<a> = PolynomialRing(QQ)
MS = MatrixSpace(SR, 3, 4)
m = MS.matrix([[1,1,1,a],
              [1,a,3,-2],
              [1,3,-1,0]])
fm(r'\[{m} \Longrightarrow {m.echelon_form()} \Longrightarrow {
   m.echelon_form().simplify_full()}\]')
```

$$\left[\begin{array}{cccc} 1 & 1 & 1 & a \\ 1 & a & 3 & -2 \\ 1 & 3 & -1 & 0 \end{array}\right] \Longrightarrow \left[\begin{array}{cccc} 1 & 0 & 0 & a + \frac{\left(a - \frac{2\,(a+2)}{a-1}\right)\left(\frac{2}{a-1} - 1\right)}{2\left(\frac{2}{a-1}+1\right)} + \frac{a+2}{a-1} \\ 0 & 1 & 0 & -\frac{a+2}{a-1} - \frac{a - \frac{2\,(a+2)}{a-1}}{(a-1)\left(\frac{2}{a-1}+1\right)} \\ 0 & 0 & 1 & \frac{a - \frac{2\,(a+2)}{a-1}}{2\left(\frac{2}{a-1}+1\right)} \end{array}\right] \Longrightarrow \left[\begin{array}{cccc} 1 & 0 & 0 & \frac{1}{2}\,a + 4 \\ 0 & 1 & 0 & -2 \\ 0 & 0 & 1 & \frac{1}{2}\,a - 2 \end{array}\right]$$

5.2 vector spaces

```
V = VectorSpace(QQ, 4)
S = V.subspace([V([-1,1,2,-2]),V([0,2,-4,2])])
U = V.subspace([V([1,-1,-1,1]),V([2,0,-4,2])])
U.basis()
U.dimension()
S.basis()
S.dimension()
```

[
(1, 0, -2, 1),
(0, 1, -1, 0)
]
2
[
(1, 0, -4, 3),
(0, 1, -2, 1)
]
2

6 boolean algebra

```
from sympy.logic import SOPform
from sympy import symbols
from sympy.logic.boolalg import truth_table

w, x, y, z = symbols('w x y z')
minterms = [5, 7, 8, 9, 10, 15]
```

```
sop = SOPform([w, x, y, z], minterms, [])

return list(truth_table(sop, [w,x,y,z]))
```

(0 0 0 0)	False
(0 0 0 1)	False
(0 0 1 0)	False
(0 0 1 1)	False
(0 1 0 0)	False
(0 1 0 1)	True
(0 1 1 0)	False
(0 1 1 1)	True
(1 0 0 0)	True
(1 0 0 1)	True
(1 0 1 0)	True
(1 0 1 1)	False
(1 1 0 0)	False
(1 1 0 1)	False
(1 1 1 0)	False
(1 1 1 1)	True

7 latex parsing and integration with emacs

apparently sagemath itself cannot parse latex, but sympy which is packaged with sagemath has an experimental latex parser which should be sufficient, apparently the parser requires a package called `antlr4-python3-runtime`, i tried installing it globally using pacman but i was still getting the following error

```
---------------------------------------------------------------------------

ImportError                               Traceback (most recent
   call last)
Input In [18], in <cell line: 1>()
----> 1 parse_latex(r"\int_a^b f(x) dx")

File /usr/lib/python3.10/site-packages/sympy/parsing/latex/
   __init__.py:35, in parse_latex(s)
    30 _latex = import_module(
    31     'sympy.parsing.latex._parse_latex_antlr',
```

```
     32     import_kwargs={'fromlist': ['X']})
     34 if _latex is not None:
---> 35     return _latex.parse_latex(s)

File /usr/lib/python3.10/site-packages/sympy/parsing/latex/
    _parse_latex_antlr.py:65, in parse_latex(sympy)
     62 antlr4 = import_module('antlr4', warn_not_installed=
        True)
     64 if None in [antlr4, MathErrorListener]:
---> 65     raise ImportError("LaTeX parsing requires the antlr4
     Python package,"
     66                       " provided by pip (antlr4-python2-
        runtime or"
     67                       " antlr4-python3-runtime) or"
     68                       " conda (antlr-python-runtime)")
     70 matherror = MathErrorListener(sympy)
     72 stream = antlr4.InputStream(sympy)

ImportError: LaTeX parsing requires the antlr4 Python package,
    provided by pip (antlr4-python2-runtime or antlr4-python3-
    runtime) or conda (antlr-python-runtime)
```

so i decided to use latex2sympy

install it using:

```
pip install latex2sympy2
```

example usage:

```
from latex2sympy2 import latex2sympy, latex2latex

tex = r"\frac{d}{dx}(x^{2}+x)"
# Or you can use '\mathrm{d}' to replace 'd'
print(latex2sympy(tex))
print(latex2latex(tex))
```

8 more resources

in sage: `https://cocalc.com/share/public_paths/6575abaa81222f77110e4e1de268ff8b1`